

EventForce Management System

Salesforce Implementation Report

K S K College of Engineering and Technology

Project environment: Event Force Management System Playground

Implementation and evidence review date: 3 October 2026

User story

As an event coordinator, I need a central place to manage clients, events, venues, vendors and feedback so that bookings remain consistent, cancellation decisions are traceable and upcoming work is visible.

Report scope

This report describes the EventForce implementation in the new Salesforce playground, following the supplied project template structure. It covers requirements, design, planning, development, security, data migration, testing and maintenance. Saved Salesforce screenshots provide implementation evidence. The test section identifies checks that require final user acceptance; configuration alone is not evidence of successful email receipt or a user login.

Contents

1 Introduction 2 Ideation 3 Requirement Analysis 4 Project Design 5 Planning and Scheduling 6 Development 7 Testing 8 Advantages 9 Conclusion and Future Scope Appendix Evidence and Source Code

1 Introduction

1.1 Project overview

EventForce centralizes the event lifecycle in Salesforce. Event records connect a client and venue; an Event Vendor junction records multiple vendor assignments. Feedback links the client to the relevant event. A Lightning app provides navigation, while reports and a dashboard summarize upcoming work.

1.2 Purpose

The project replaces scattered booking details with related records, automates reminders and cancellation processing, prevents conflicting venue bookings and controls access by responsibility. It demonstrates Salesforce configuration and Apex development within a dedicated educational playground.

1.3 Environment and boundaries

The implementation uses `playful-unicorn-d2io9f-dev-ed.trailblaze.lightning.force.com`. The earlier org is outside this project environment. Sample records are labelled demonstrations or samples. No production migration has been performed; the guide treats deployment as preparation in a standalone Developer Edition environment.

2 Ideation Phase

2.1 Problem statement

User	Problem	Expected outcome
Event coordinator	Booking information is scattered across clients, venues and vendors.	One linked event record and a consistent workflow.
Vendor manager	Event assignments and vendor information are difficult to coordinate.	Vendor records and event visibility with limited editing.
Client	Reminders and cancellation updates can be missed.	Automated notification workflows and feedback collection.
Event administrator	Conflicting bookings and informal cancellations reduce reliability.	Validation, controlled approvals and reporting.

2.2 Empathy map

The coordinator thinks about avoiding missed commitments, hears changing client requirements, sees venue and vendor constraints, and spends time updating schedules. The principal pains are duplicate information and booking conflicts. The desired gains are clear ownership, timely reminders and visible approval outcomes. This persona is a design assumption, not a claim of a conducted customer interview.

2.3 Ideas and prioritization

Manual registers and spreadsheets are simple to start but require repeated coordination. A separate custom portal adds development and maintenance work. Salesforce was selected because its related data model, declarative automation, Apex, access controls and reporting support the project requirements within one platform.

3 Requirement Analysis Phase

3.1 Customer journey

Client enquiry and registration → event planning → venue selection → vendor assignment → confirmation and reminder scheduling → event execution → feedback → operational reporting. A cancellation request branches into approval or rejection and a recorded final status.

3.2 Functional requirements

Requirement	Implementation
Manage core information	Event, Client, Vendor, Venue and Feedback custom objects.
Assign multiple vendors	EventVendor junction with two master detail relationships.
Prevent conflicting bookings	Apex validates venue and date before saving.
Manage cancellation	Approval process with Canceled and Rejected final status updates.
Notify clients	Cancellation notification flow and three-day reminder flow.
Complete past events	Scheduled BatchCompleteEvents at 8 PM daily.
Control access	Four role/profile combinations, Feedback Manager and sharing rules.
Monitor operations	Upcoming Events by Month report and Operations Dashboard.

3.3 Nonfunctional requirements

The solution must maintain related-record integrity, restrict record visibility, process bulk records, present understandable validation messages and remain maintainable through documented metadata and tests. No measured load benchmark or production availability guarantee is claimed.

4 Project Design Phase

4.1 Data model

Object	Purpose	Relationships
Event__c	Name, type, date, status and formula budget	Lookup to Client and Venue
Client__c	Contact and location information	Referenced by Event and Feedback
Vendor__c	Services, contact and availability	Master of EventVendor
Venue__c	Address, website and availability	Referenced by Event
Feedback__c	Autonumber, rating and comments	Lookup to Client and Event
EventVendor__c	Event to vendor assignment	Master detail to Event and Vendor

4.2 Architecture and technology

Salesforce Lightning provides the user interface. Custom objects store data. Formula fields and validation rules handle declarative calculations and checks. Record-triggered flows send notifications; the approval process controls cancellation decisions. Apex handles booking conflicts, venue synchronization and scheduled completion. Reports and dashboards summarize stored data.

4.3 Implementation choices

The app is named EventForce for the requested distinct presentation. The guide demonstrates separate helper and trigger examples; this implementation consolidates venue synchronization and booking validation in EventForceService and EventForceTrigger. The report uses the deployed names so that the source code and demonstration remain traceable.

5 Project Planning and Scheduling

The implementation was organized around the guide's five phases. The following sequence is a work breakdown, not a fabricated record of team sprint velocity or individual contributions.

Phase	Work package	Completion evidence
Requirements and planning	Identify objects, users, relationships and workflows	Requirements and data model in this report
Backend configuration	Fields, formula, validation, Apex, approval and flows	Setup screenshots and Apex tests
UI customization	Tabs, app, monthly report and dashboard	Application and analytics screenshots
Data testing and security	Sample imports, profiles, roles, permissions and sharing	Import results, saved settings and test results
Deployment and maintenance	Keep source, evidence, report and demo narration	Local project files and this report

Priority and acceptance

Booking correctness and related data come first; notification and cancellation behavior follow. Security is checked before user demonstrations. Documentation records the implemented behavior and any remaining acceptance checks. A final submission should include the report and a screen recording narrated by the student.

6 Development Phase

6.1 Requirement analysis and planning

The new playground separates this implementation from the earlier org. The project maps Client, Coordinator, Vendor Manager and Event Admin responsibilities to related data and controlled workflows.

6.2 Objects and fields

Six custom objects implement the data model. Event Type drives a Currency formula budget. Event Status tracks progression, cancellation and rejection. Client email validation and the Feedback event lookup filter protect data quality. Feedback uses an autonumber record name.

The screenshot shows the Salesforce Developer Edition interface for the 'Event' object. The left sidebar lists various setup options, with 'Fields & Relationships' selected. The main area displays a table of fields and relationships for the 'Event' object, sorted by field label. The table has columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed. There are 10 items listed.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Client	Client_c	Lookup(Client)		✓
Created By	CreatedById	Lookup(User)		
Event Budget	Event_Budget_c	Formula (Currency)		
Event Date	Event_Date_c	Date		
Event Name	Name	Text(80)		✓
Event Status	Event_Status__c	Picklist		
Event Type	Event_Type__c	Picklist		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User,Group)		✓
Venue	Venue__c	Lookup(Venue)		✓

Event fields and relationships captured from the project playground.

The screenshot shows the Salesforce Developer Edition interface for the 'Event Vendor' object. The left sidebar lists various setup options, with 'Fields & Relationships' selected. The main area displays a table of fields and relationships for the 'Event Vendor' object, sorted by field label. The table has columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed. There are 3 items listed.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Event Vendor Name	Name	Text(80)		✓
Last Modified By	LastModifiedById	Lookup(User)		
Vendor	Vendor__c	Master-Detail(Vendor)		✓

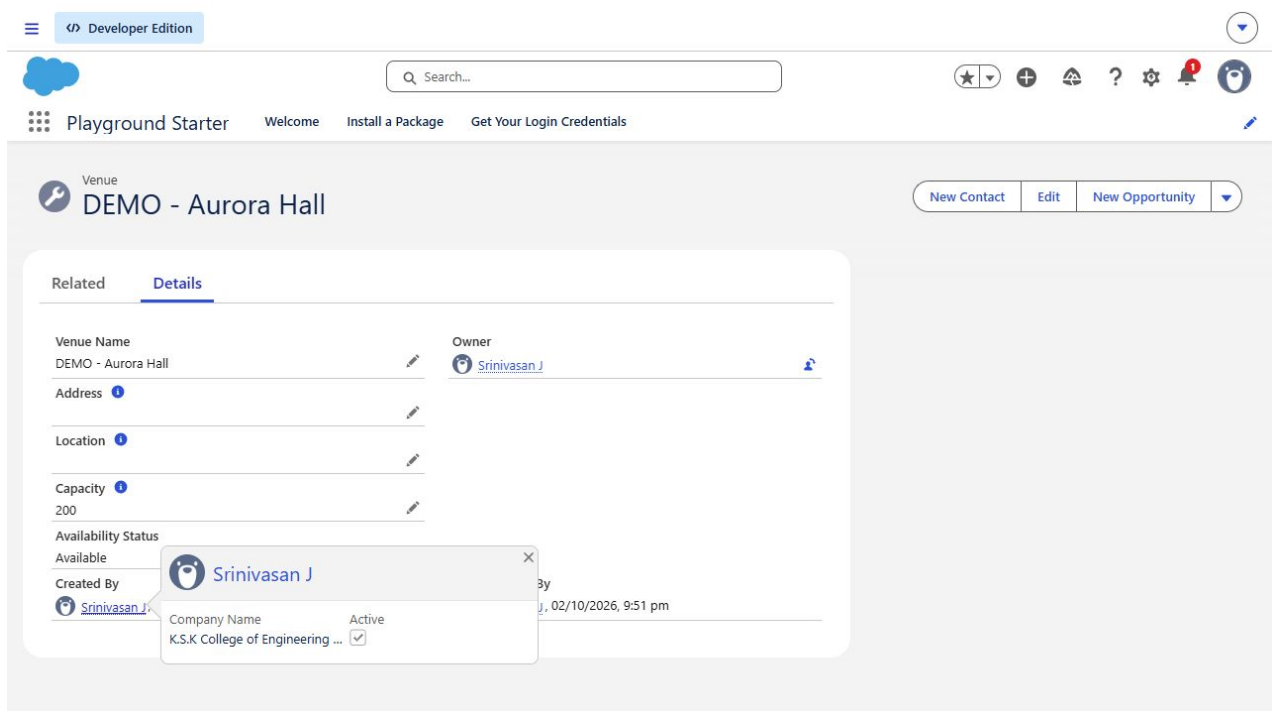
Event Vendor junction relationships.

6.3 Booking Logic and Venue Availability

EventForceTrigger delegates to EventForceService. The service checks venue/date conflicts across stored records and records in the same operation. Venue rows are locked during validation to serialize competing bookings. Confirmed and Pending Cancellation events require client, venue and event date.

Venue synchronization reserves venues referenced by Confirmed or Pending Cancellation events and releases them when no such event remains. Venues marked Not Available retain that state. Canceled, Rejected and Completed records do not block a new booking in this implementation.

These rules are more explicit than the guide's short example code. Planned events participate in conflict prevention, while reservation status is driven by Confirmed and Pending Cancellation events. This distinction should be explained during the demonstration.



A venue is Available after the cancellation demonstration releases its reservation.

6.4 Cancellation Approval

The active Event Cancellation Process accepts events with Pending Cancellation status. The event owner submits the request and the configured current user acts as the demonstration approver. Submission sends an alert. Approval updates the status to Canceled; rejection updates it to Rejected, as required by the guide's field-update step.

Approval and rejection were exercised through separate demonstration records. Approval History provides the decision trail. The client cancellation flow is separate from the approval process and runs when a record changes to Canceled with a related client email.

The screenshot displays the 'DEMO - Cancellation Approval Test' record in the EventForce Management System. The interface includes a top navigation bar with 'Developer Edition' and a search bar. The main content area shows a list of sections: 'Event Vendors (0)', 'Feedbacks (0)', and 'Approval History (2)'. The 'Approval History' section contains a table with the following data:

Step Name	Date	Status	Assigned To
Review Event Cancellation	02/10/2026, 9:51 pm	Approved	Srinivasan J
Approval Request Submi...	02/10/2026, 9:50 pm	Submitted	Srinivasan J

Below the table is a 'View All' link. To the right of the main content area, there is a sidebar with filters and a 'Show All Activities' button.

Completed approval history from the cancellation demonstration.

The screenshot displays the 'DEMO - Cancellation Rejection Test' record in the EventForce Management System. The interface is similar to the previous one, but the 'Approval History' section shows a different status for the first step. The table data is as follows:

Step Name	Date	Status	Assigned To
Review Event Cancellation	02/10/2026, 9:53 pm	Rejected	Srinivasan J
Approval Request Submi...	02/10/2026, 9:53 pm	Submitted	Srinivasan J

Below the table is a 'View All' link. The sidebar on the right remains the same.

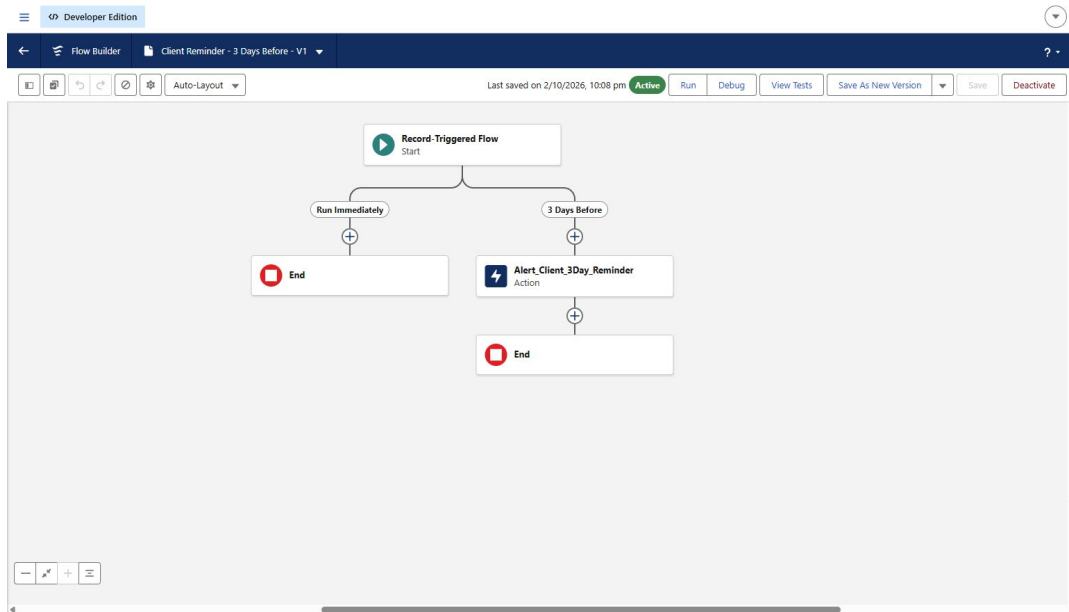
Completed rejection history from the rejection demonstration.

6.5 Notification Flows

Client Reminder - 3 Days Before is active. Confirmed events with date, client and email qualify. Its scheduled path uses Event Date with a three-day offset. The email goes to the client and copies the event owner. Debugging validated recipient resolution and merged event, client and venue information.

A separate cancellation notification flow sends the client an update when Event Status changes to Canceled. Entry checks avoid blank recipients and repeated notifications from unrelated edits.

The reminder demonstration for 20 October 2026 produced a scheduled path dated 17 October 2026. This verifies scheduling, not inbox delivery. Actual receipt remains a user acceptance check.



Active reminder flow.

Time-Based Automations

Monitor the queue of pending automations. To filter the list, enter criteria before you click Search. The queue includes scheduled paths for record-triggered flows, schedule-triggered flows, time-dependent workflow actions, scheduled actions for processes built in Process Builder, and flow resume events.

Record Name	Object	Workflow Rule, Flow, or Process Name	Scheduled Date	Created By	Created Date	Automation Type
☐ DEMO - Confirmed Reminder Test	Event	Client_Reminder_3_Days_Before	17/10/2026, 12:00 am	J. Srinivasan	02/10/2026, 10:11 pm	Flow Scheduled Path

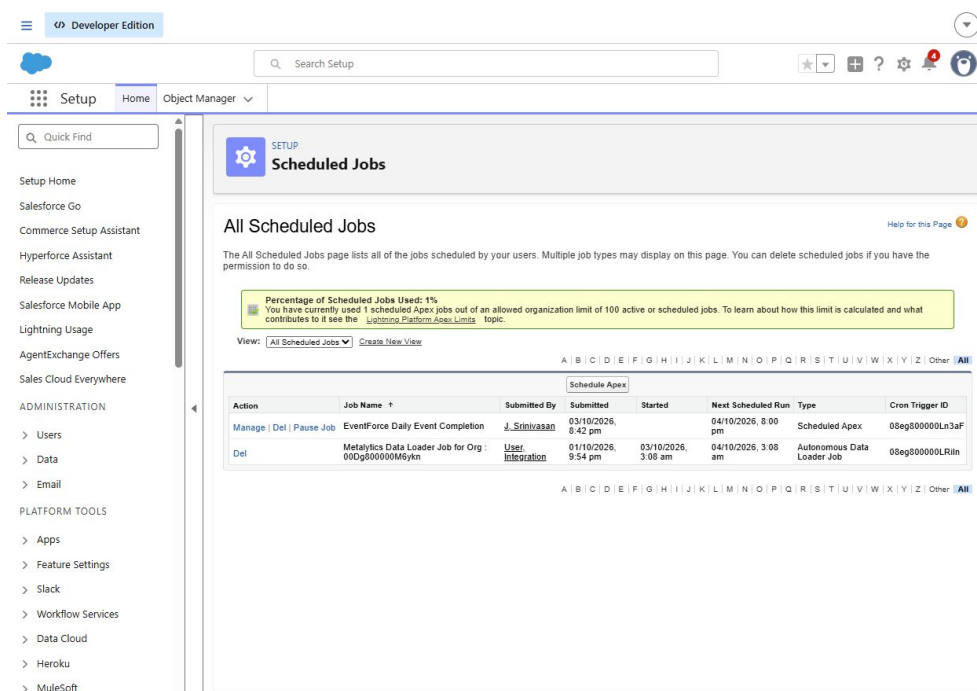
Scheduled path evidence showing the three-day offset.

6.6 Scheduled Apex

BatchCompleteEvents implements Database.Batchable and Schedulable. To follow the guide literally, it selects events where Event Date is earlier than today and status is not Completed, then sets status to Completed. The daily job uses cron 0 0 20 * * ?, corresponding to 8 PM in the scheduling user's configured time zone.

This rule includes past Planned, Canceled and Rejected events. Consequently, a canceled event's current status can later become Completed even though its approval history remains. This is the guide's specified batch behavior and should be considered before any real production adoption.

The schedule was replaced after Salesforce required removal of the pending job before editing its class. The replacement retained the original daily time. Earlier execution on 3 October completed with zero failures and no eligible batches; automated tests separately exercise eligible records.



The screenshot shows the Salesforce Setup interface. The left sidebar contains navigation links for Setup Home, Salesforce Go, Commerce Setup Assistant, Hyperforce Assistant, Release Updates, Salesforce Mobile App, Lightning Usage, AgentExchange Offers, Sales Cloud Everywhere, ADMINISTRATION (Users, Data, Email), PLATFORM TOOLS (Apps, Feature Settings, Slack, Workflow Services, Data Cloud, Heroku, MuleSoft), and a search bar. The main content area is titled 'Scheduled Jobs' and displays 'All Scheduled Jobs'. A yellow banner indicates 'Percentage of Scheduled Jobs Used: 1%' and provides a link to 'Lightning Platform Apex Limits'. Below this, a table lists scheduled jobs. The table has columns for Action, Job Name, Submitted By, Submitted, Started, Next Scheduled Run, Type, and Cron Trigger ID. Two jobs are listed: 'EventForce Daily Event Completion' and 'Metalitics Data Loader Job for Org: 00Dg000000M6ykn'.

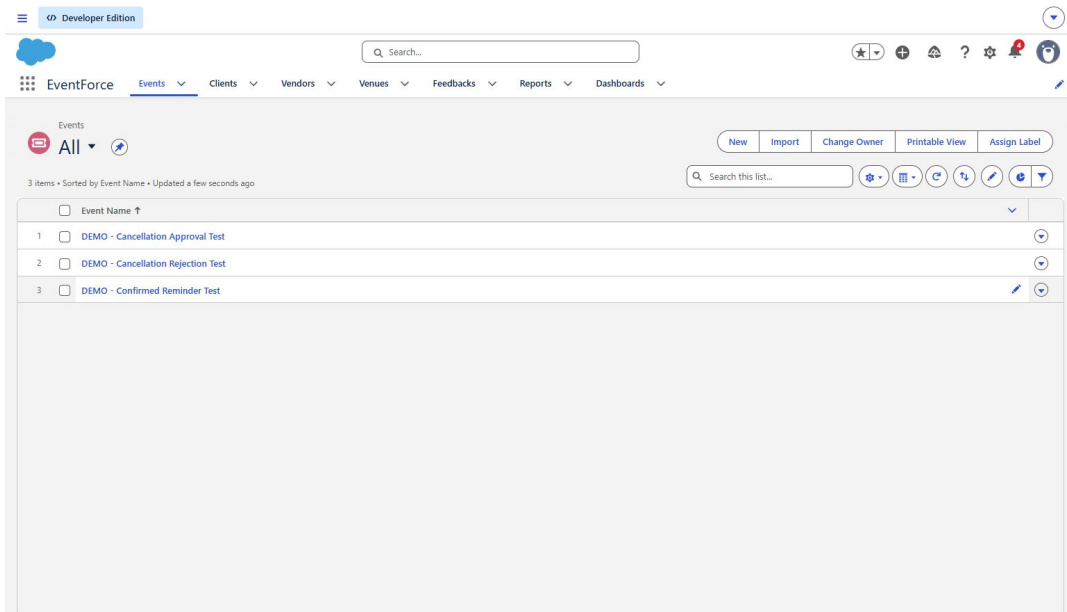
Action	Job Name	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	EventForce Daily Event Completion	J. Srinivasan	03/10/2026, 8:42 pm		04/10/2026, 8:00 pm	Scheduled Apex	08eg000000Ln3aF
Del	Metalitics Data Loader Job for Org: 00Dg000000M6ykn	User Integration	01/10/2026, 9:54 pm	03/10/2026, 3:08 am	04/10/2026, 3:08 am	Autonomous Data Loader Job	08eg000000LRln

Restored daily schedule following the guide-aligned batch change.

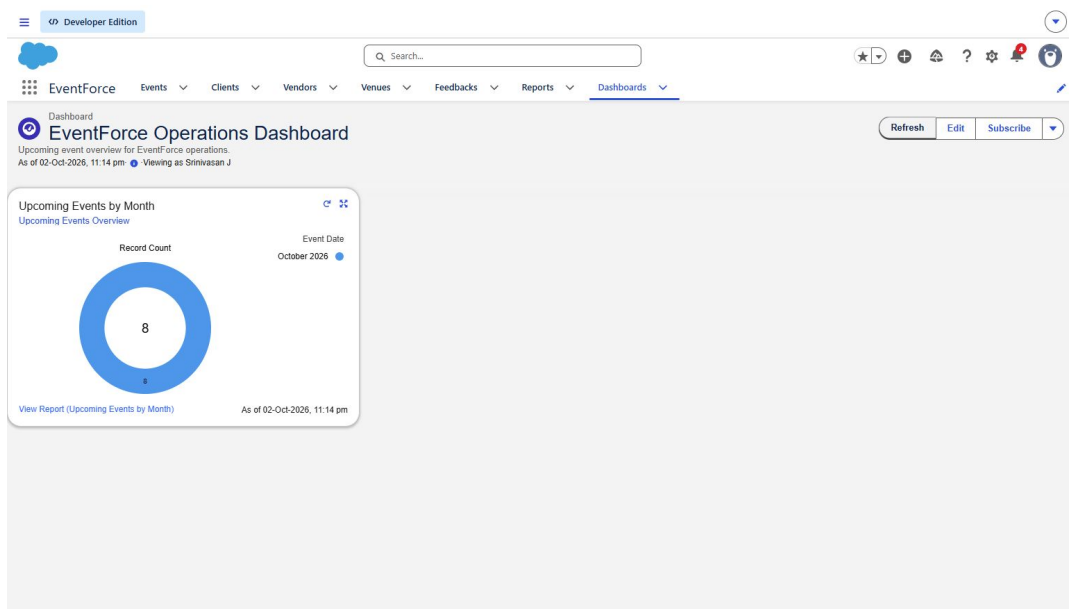
6.7 User Interface and Analytics

The EventForce Lightning app includes Events, Clients, Vendors, Venues, Feedbacks, Reports and Dashboards. Five custom tabs use distinct icons. The monthly report selects all events in the next 30 days except Canceled records, groups Event Date by calendar month and summarizes record count and budget.

The EventForce Operations Dashboard uses the monthly report with a donut chart, six maximum displayed values, the subtitle Upcoming Events Overview and a light widget theme. Report results depend on the current date, so demonstration counts can change.



EventForce application and navigation.



Operations dashboard captured during implementation.

6.8 Data Migration

Sample CSV files were prepared for all six objects. Parent records were imported before related records; relationships were matched by names. The Data Import Wizard results reported zero failed rows for all six imports. Fictional sample email addresses and labelled records distinguish demonstrations from real operational data.

Object	Records represented after import
Client	10 including the existing demo client
Venue	10 including the existing demo venue
Vendor	10
Event	13 including three demonstration events
Feedback	10
EventVendor	10

These counts describe the verified import checkpoint. Later intentional test activity may change totals. Feedback inserts should not be repeated casually because they do not have a matching duplicate key in the prepared import.

The screenshot shows the Salesforce Data Import Wizard interface. At the top, there's a navigation bar with 'Developer Edition' and a search bar. Below the navigation bar, the 'Data Import Wizard' section is active. It displays a table titled 'Recent Import Jobs' with columns: Status, Object, Records Created, Records Updated, Records Failed, Start Date, and Processing Time (ms). The table lists five completed imports for Event Vendor, Feedback, Event, Vendor, and Venue, all with zero failed records. Below the table is a 'Bulk API Monitoring' button. At the bottom, there's a section titled 'Before you import your data...' with three tips: 'Clean up your data import file', 'Make sure your field names match Salesforce field names', and 'Don't import too many records at once'. The interface also includes a 'Help for this page' link and a 'Collapse' button for the tips section.

Status	Object	Records Created	Records Updated	Records Failed	Start Date	Processing Time (ms)
Closed	Event Vendor	10	0	0	10-02-2026 05:28	179
Closed	Feedback	10	0	0	10-02-2026 05:25	183
Closed	Event	10	0	0	10-02-2026 05:23	319
Closed	Vendor	10	0	0	10-02-2026 05:21	205
Closed	Venue	9	0	0	10-02-2026 05:19	145

Before you import your data...

- Clean up your data import file**
You'll have fewer errors to resolve if your data file is clean and free of duplicates. [Watch video](#)
- Make sure your field names match Salesforce field names**
You'll be required to map your data fields to Salesforce data fields. Data in unmapped fields is not imported. [View a list of Salesforce data fields](#).
- Don't import too many records at once**
Using the Data Import Wizard, import up to 50,000 records at a time. Importing too many records can slow down your org for all users, especially during periods of peak usage.

Import your data in 3 easy steps!

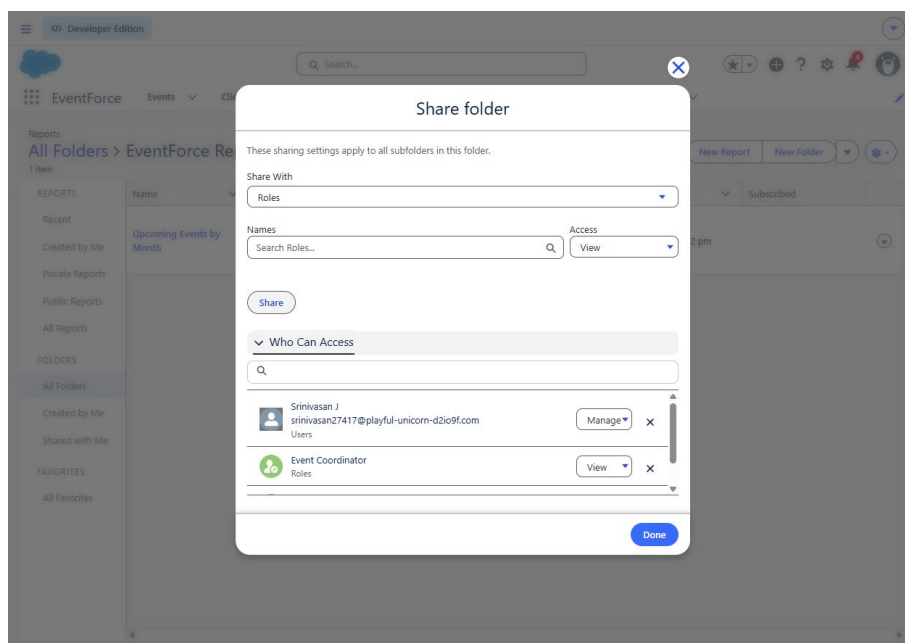
Data import evidence.

6.9 Security Configuration

Profile	Object permissions
Event Admin	Administrator profile clone
Event Coordinator	Event CRED; Client Read/Edit; Vendor and Venue Read; Feedback Create/Read
Vendor Manager	Vendor CRED; Event, Venue, Client and Feedback Read
Client Profile	Event Read; Feedback Create/Read; no Vendor or Venue access

CRED means Create, Read, Edit and Delete. Feedback Manager additionally grants these four Feedback permissions to the coordinator. The three operational profiles use the guide's eight-character minimum, two-hour inactivity setting and non-expiring password policy; actual role-session acceptance remains to be checked.

Event internal and external defaults are Private. Event_Sharing_For_Vendors gives the Vendor Manager role Read Only access to events owned by users in the Event Coordinator role. The report folder gives View access to those two roles. Event Admin reports to CEO; Coordinator, Vendor Manager and Client are below Event Admin.

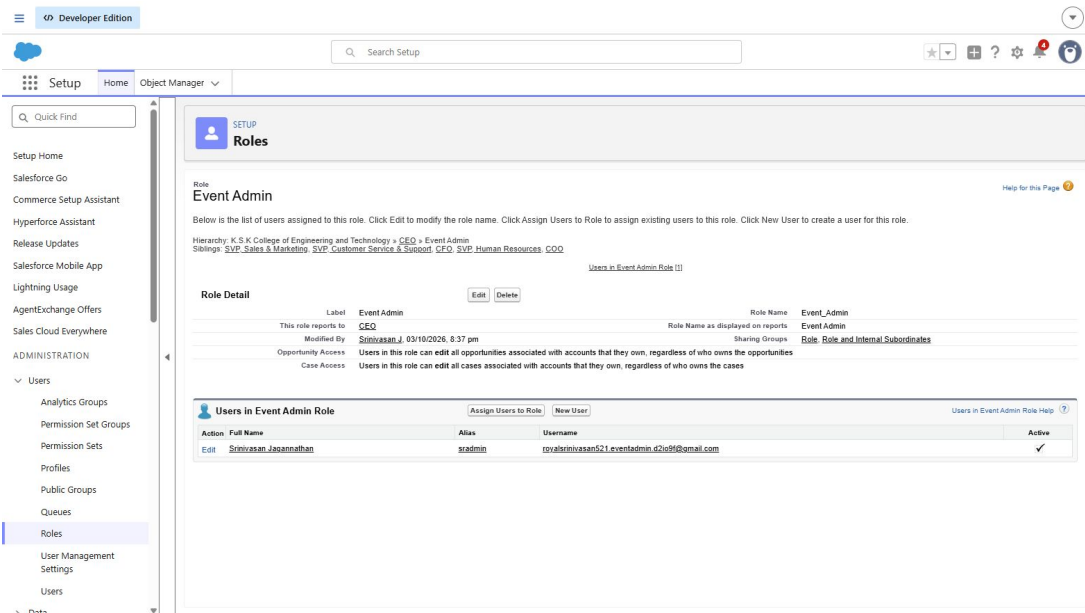


Saved report folder shares for the two required roles.

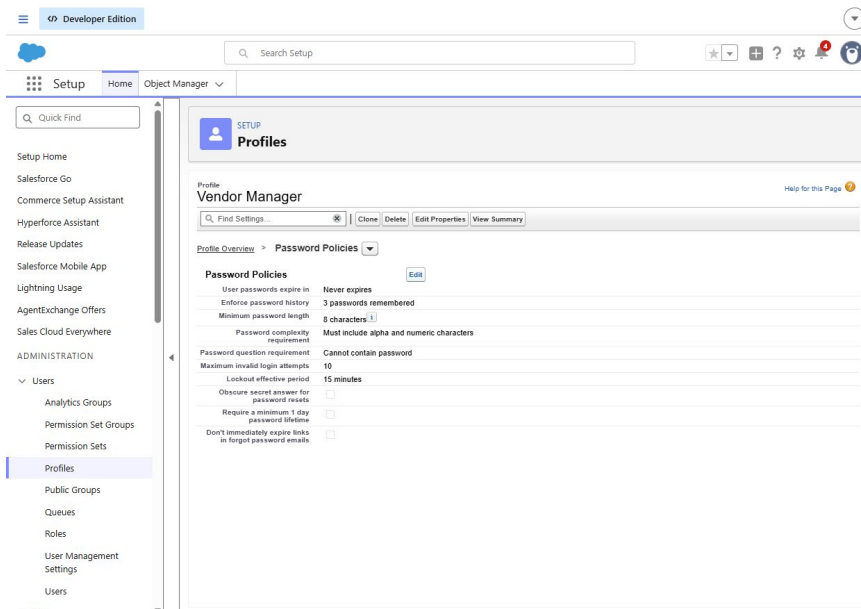
6.10 Deployment and Maintenance

The guide does not require deployment into a production org. The educational delivery remains in the new playground. Local metadata and sample CSVs accompany implementation evidence. They support review and future packaging, but are not represented as a fresh complete metadata export of every manually configured component.

Maintenance consists of monitoring Apex Jobs and scheduled paths, checking failed flow interviews and email delivery, reviewing access after user changes, and running regression tests after schema or Apex edits. Any future production release would require a separate environment, complete metadata retrieval and a deployment-specific test and security review.



Saved Event Admin parent role is CEO.



Vendor Manager password policy saved as specified by the guide.

7 Functional and Performance Testing

Check	Observed result	Evidence or limit
Apex compilation	Successful	Invalid classes recompiled on 3 October
Booking and venue tests	4 methods passed	EventForceServiceTest
Batch tests after guide scope edit	2 methods passed	Updated regression on 3 October
Cancellation approval	Passed	Canceled status and approval history
Cancellation rejection	Passed	Rejected status and rejection history
Reminder scheduling	Passed	Three-day offset visible in queue
Sample imports	Passed	Zero failed rows in six import jobs
Scheduled execution	Completed with zero failures	No eligible records in that execution
Email inbox receipt	Not verified	A scheduled path or successful action does not prove receipt
Four user login and access checks	Not verified	Accounts and assignments verified; separate sessions not exercised
Load and concurrency benchmark	Not measured	No throughput claim

The test evidence supports the specific results shown. It is not a guarantee of zero defects. The updated batch regression passed after the guide-aligned query change.

8 Advantages and 9 Conclusion

8.1 Advantages

Related records reduce duplicate data entry and help users follow an event from client request to feedback. Automated booking checks reduce conflicting venue allocations. Approval History records cancellation decisions. Scheduled reminders and completion processing reduce repeated manual steps, while reporting makes upcoming work easier to inspect.

9.1 Conclusion

The playground implements the main EventForce workflow using Salesforce configuration and Apex. Security corrections, actual approval demonstrations, import results and regression evidence make the implementation reviewable. The remaining acceptance checks are actual email receipt and role-specific login behavior. These must not be presented as passed without observation.

9.2 Future scope

Future work could add client self-service, richer vendor assignment notifications, calendar-based venue visibility, mobile-specific layouts and integrations. A production version should revisit the guide's non-expiring passwords and broad past-event completion rule. These are future improvements, not features claimed in this educational implementation.

References

Supplied project references: EVENTFORCE MANAGEMENT SYSTEM- SALESFORCE IMPLEMENTATION NM.pdf and EVENTFORCE- SALESFORCE IMPLEMENTATION.pdf.

Implementation evidence: screenshots from the Event Force Management System Playground, captured during October 2026. The template's unrelated vehicle/dealer testing language has been replaced with EventForce-specific results.

Appendix Source Code

The following local source reflects the implemented Apex components. Salesforce remains the authority for manually configured metadata. Test classes are included in the accompanying project source folder.

BatchCompleteEvents.cls

```
public without sharing class BatchCompleteEvents implements Database.Batchable<SObject>, Schedulable
{
    public Database.QueryLocator start(Database.BatchableContext context) {
        return Database.getQueryLocator([SELECT Id, Event_Status__c FROM Event__c WHERE
Event_Date__c < TODAY AND Event_Status__c != 'Completed']);
    }
    public void execute(Database.BatchableContext context, List<Event__c> scope) {
        for (Event__c e : scope) e.Event_Status__c = 'Completed';
        update scope;
    }
    public void finish(Database.BatchableContext context) {}
    public void execute(SchedulableContext context) {
        Database.executeBatch(new BatchCompleteEvents(), 200);
    }
}
```

EventForceService.cls

```
public without sharing class EventForceService {
    private static Boolean blocks(Event__c e) {
        return e.Event_Status__c != 'Canceled' && e.Event_Status__c != 'Rejected' &&
e.Event_Status__c != 'Completed';
    }
    public static void validateBookings(List<Event__c> records) {
        Set<Id> venues = new Set<Id>();
        Set<Id> ids = new Set<Id>();
        Set<Date> dates = new Set<Date>();
        for (Event__c e : records) {
            if (e.Id != null) ids.add(e.Id);
            if (e.Venue__c != null) venues.add(e.Venue__c);
            if (e.Event_Date__c != null) dates.add(e.Event_Date__c);
            if ((e.Event_Status__c == 'Confirmed' || e.Event_Status__c == 'Pending Cancellation') &&
(e.Venue__c == null || e.Client__c == null || e.Event_Date__c == null)) {
                e.addError('A confirmed event needs a client, venue, and event date.');
            }
        }
        if (venues.isEmpty()) return;
        // Lock shared venues to serialize competing booking transactions.
        List<Venue__c> locked = [SELECT Id FROM Venue__c WHERE Id IN :venues FOR UPDATE];
        Set<String> booked = new Set<String>();
        for (Event__c e : [SELECT Id, Venue__c, Event_Date__c, Event_Status__c FROM Event__c WHERE
Venue__c IN :venues AND Event_Date__c IN :dates AND Id NOT IN :ids]) {
            if (blocks(e)) booked.add(String.valueOf(e.Venue__c) + ':' +
String.valueOf(e.Event_Date__c));
        }
        for (Event__c e : records) {
            if (!blocks(e) || e.Venue__c == null || e.Event_Date__c == null) continue;
            String key = String.valueOf(e.Venue__c) + ':' + String.valueOf(e.Event_Date__c);
            if (booked.contains(key)) e.addError('This venue is already booked for the selected
event date.');
            else booked.add(key);
        }
    }
    public static void syncVenues(List<Event__c> records, List<Event__c> previous) {
        Set<Id> venues = new Set<Id>();
        if (records != null) for (Event__c e : records) if (e.Venue__c != null)
venues.add(e.Venue__c);
        if (previous != null) for (Event__c e : previous) if (e.Venue__c != null)
venues.add(e.Venue__c);
        if (venues.isEmpty()) return;
        Set<Id> reserved = new Set<Id>();
        for (AggregateResult row : [SELECT Venue__c venue FROM Event__c WHERE Venue__c IN :venues
AND Event_Status__c IN ('Confirmed', 'Pending Cancellation') GROUP BY Venue__c]) {
            reserved.add((Id) row.get('venue'));
        }
        List<Venue__c> changes = new List<Venue__c>();
        for (Venue__c v : [SELECT Id, Availability_Status__c FROM Venue__c WHERE Id IN :venues]) {
            if (v.Availability_Status__c == 'Not Available') continue;
            String desired = reserved.contains(v.Id) ? 'Reserved' : 'Available';
            if (v.Availability_Status__c != desired) {
                v.Availability_Status__c = desired;
                changes.add(v);
            }
        }
        if (!changes.isEmpty()) update changes;
    }
}
```